

Phase 1 – API Test Report

Phase 1 – APIs Tested

- POST /api/auth/register
- POST /api/auth/login
- GET /api/auth/me
- PATCH /api/auth/me
- PUT /api/auth/change-password

Phase 1 – Test Coverage

- Successful Customer Registration.
- Duplicate Email Registration.
- Invalid Registration Request.
- Successful Login.
- Invalid Email or Password.
- Blocked Account Login.
- Deactivated Account Login.
- Missing Login Fields.
- Empty Login Request.
- Valid JWT Access.
- Missing JWT Token.
- Invalid JWT Token.
- Expired JWT Token.
- Access to Public Endpoints without JWT.
- Successful Current Profile Retrieval.
- Successful First Name Update.
- Successful Last Name Update.
- Successful Phone Update.
- Successful Multiple-Field Update.
- Empty Profile Update Request.
- Rejection of Unsupported or Immutable Fields.
- Successful Password Change.
- Incorrect Old Password.
- Invalid Password Change Request.
- Login Failure using the Old Password after Change.
- Login Success using the New Password after Change.
- Verification that Password and Password Hash are never exposed in API Responses.

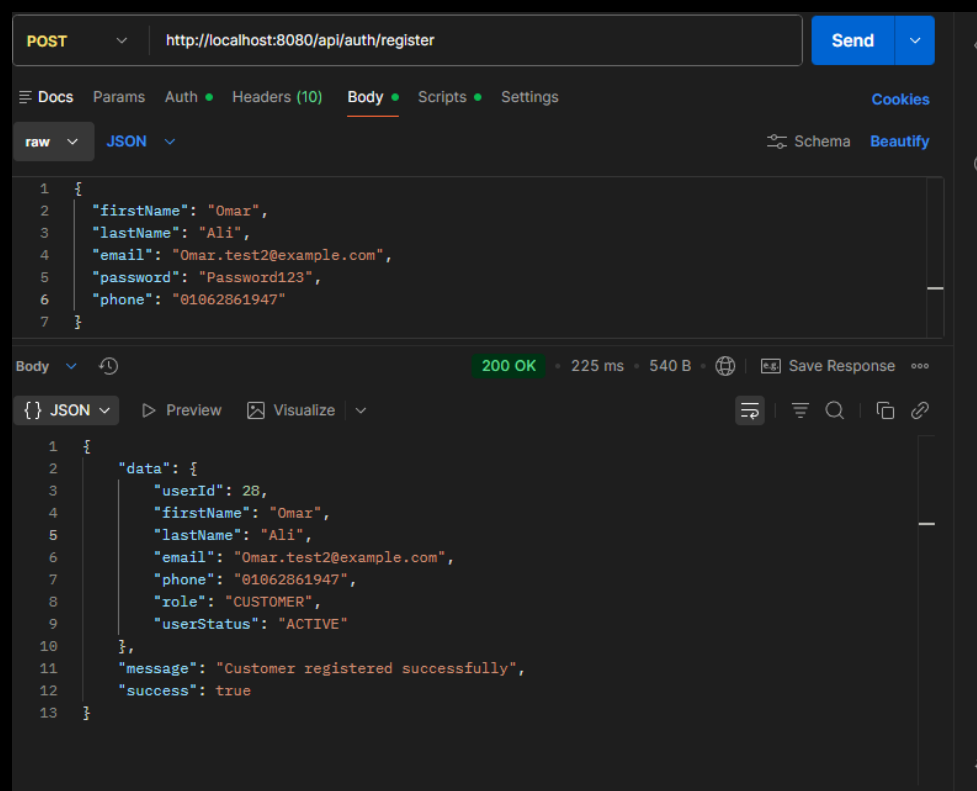
F1.2 – Register Customer: Successful Registration

Endpoint: POST /api/auth/register

Access: Public

Scenario: Registering a new customer with valid data and a unique email.

Result: 200 OK – The customer account is created with the CUSTOMER role and ACTIVE status.



F1.2 – Register Customer: Duplicate Email

Endpoint: POST /api/auth/register

Access: Public

Scenario: Registering a customer using an email that already exists.

Result: 409 Conflict — The request is rejected with the DUPLICATE_EMAIL error code.

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8080/api/auth/register
- Body (raw):**

```
1 {
2   "firstName": "Omar",
3   "lastName": "Ali",
4   "email": "Omar.test2@example.com",
5   "password": "Password123",
6   "phone": "01062861947"
7 }
```
- Status:** 409 Conflict (11 ms, 433 B)
- Body (JSON):**

```
1 {
2   "details": [],
3   "errorCode": "DUPLICATE_EMAIL",
4   "message": "Email already exists",
5   "success": false
6 }
```

F1.3 — Login: Successful Authentication

Endpoint: POST /api/auth/login

Access: Public

Scenario: Logging in with a valid email and password.

Result: 200 OK — A JWT access token, Bearer token type, and authenticated user data are returned successfully.

The screenshot displays a REST client interface with the following details:

- Request:**
 - Method: POST
 - URL: http://localhost:8080/api/auth/login
 - Body (raw):

```
1 {
2   "email": "mohammed.login.test@example.com",
3   "password": "Test@12345"
4 }
```
- Response:**
 - Status: 200 OK
 - Time: 1.21 s
 - Size: 768 B
 - Body (JSON):

```
1 {
2   "data": {
3     "accessToken": "eyJhbGciOiJIUzI1NiJ9.eyJyY2x1Ijo1Q1VTE9NRViiLCJzdmIiOiJtb2hhbW1lZC5sb2dpbi50ZXN0QGV4YW1wbGUyY29tIiwiaWF0IjoxNzgzMTMjYz0TIxLCJleHAiOjE3ODUyNjU3MjF9.
4     z4z0f6VReeCr815Un0p151zW2JvNmKu5KBSKXdwjKFI",
5     "tokenType": "Bearer",
6     "user": {
7       "userId": 61,
8       "firstName": "Mohammed",
9       "lastName": "Ali",
10      "email": "mohammed.login.test@example.com",
11      "role": "CUSTOMER",
12      "userStatus": "ACTIVE",
13      "airlineId": null
14     }
15   },
16   "message": "Login successful",
17   "success": true
18 }
```

[F1.3 — Login: Invalid Credentials](#)

Endpoint: POST /api/auth/login

Access: Public

Scenario: Logging in with an invalid email or password.

Result: 401 Unauthorized — The request is rejected with the INVALID_CREDENTIALS error code without revealing which credential is incorrect.

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8080/api/auth/login
- Body Type:** raw
- Request Body:**

```
1 {
2   "email": "unknownuser@example.com",
3   "password": "Test@12345"
4 }
```
- Response:** 401 Unauthorized (24 ms, 446 B)
- Response Body (JSON):**

```
1 {
2   "details": [],
3   "errorCode": "INVALID_CREDENTIALS",
4   "message": "Invalid email or password",
5   "success": false
6 }
```

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8080/api/auth/login
- Body Type:** raw
- Request Body:**

```
1 {
2   "email": "mohammed-email",
3   "password": "Test@12345"
4 }
```
- Response:** 400 Bad Request (26 ms, 426 B)
- Response Body (JSON):**

```
1 {
2   "details": [
3     "Email must be valid"
4   ],
5   "errorCode": "VALIDATION_ERROR",
6   "message": "Validation failed",
7   "success": false
8 }
```

F1.3 — Login: Deactivated Account

Endpoint: POST /api/auth/login

Access: Public

Scenario: Logging in with valid credentials for a deactivated user account.

Result: 403 Forbidden — Login is rejected with the ACCOUNT_DEACTIVATED error code.

```
UPDATE USERS
SET USER_STATUS = 'DEACTIVATED'
WHERE EMAIL = 'mohammed.login.test@example.com';
```

The screenshot displays a REST client interface with a POST request to `http://localhost:8080/api/auth/login`. The request body is a JSON object: `{ "email": "mohammed.login.test@example.com", "password": "Test@12345" }`. The response is a 403 Forbidden status with a response time of 119 ms and a body size of 440 B. The response body is shown in JSON format: `{ "details": [], "errorCode": "ACCOUNT_DEACTIVATED", "message": "Account is deactivated", "success": false }`. The interface includes tabs for Docs, Params, Authorization, Headers (11), Body, Scripts, and Settings. The Body tab is active, showing the raw JSON request and response. The response status is highlighted in orange, and the response body is displayed in a light blue box.

```
POST http://localhost:8080/api/auth/login
```

Send

Docs Params Authorization Headers (11) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Schema Beautify

```
1 {
2   "email": "mohammed.login.test@example.com",
3   "password": "Test@12345"
4 }
```

Body Cookies (1) Headers (11) Test Results (0/1)

403 Forbidden 119 ms 440 B Save Response

JSON Preview Try with different auth credentials

```
1 {
2   "details": [],
3   "errorCode": "ACCOUNT_DEACTIVATED",
4   "message": "Account is deactivated",
5   "success": false
6 }
```

F1.3 — Login: Blocked Account

Endpoint: POST /api/auth/login

Access: Public

Scenario: Logging in with valid credentials for a blocked user account.

Result: 403 Forbidden — Login is rejected with the ACCOUNT_BLOCKED error code.

```
UPDATE USERS
SET USER_STATUS = 'BLOCKED'
WHERE EMAIL = 'mohammed.login.test@example.com';
```

The screenshot displays a REST client interface with a POST request to `http://localhost:8080/api/auth/login`. The request body is a JSON object: `{ "email": "mohammed.login.test@example.com", "password": "Test@12345" }`. The response is a 403 Forbidden status with a response time of 123 ms and a body size of 432 B. The response body is a JSON object: `{ "details": [], "errorCode": "ACCOUNT_BLOCKED", "message": "Account is blocked", "success": false }`.

```
POST http://localhost:8080/api/auth/login

{
  "email": "mohammed.login.test@example.com",
  "password": "Test@12345"
}
```

403 Forbidden • 123 ms • 432 B • Save Response

```
{
  "details": [],
  "errorCode": "ACCOUNT_BLOCKED",
  "message": "Account is blocked",
  "success": false
}
```

F1.3 — Login: Missing Password Validation

Endpoint: POST /api/auth/login

Access: Public

Scenario: Submitting a login request with an empty password or Email.

Result: 400 Bad Request — The request is rejected with the VALIDATION_ERROR code and the message “Password is required.”

POST http://localhost:8080/api/auth/login

Body: raw

```
1 {
2   "email": "mohammed.login.test@example.com",
3   "password": ""
4 }
```

400 Bad Request • 8 ms • 427 B • Save Response

Body: JSON

```
1 {
2   "details": [
3     "Password is required"
4   ],
5   "errorCode": "VALIDATION_ERROR",
6   "message": "Validation failed",
7   "success": false
8 }
```

POST http://localhost:8080/api/auth/login

Body: raw

```
1 {
2   "email": "",
3   "password": "Test@12345"
4 }
```

400 Bad Request • 10 ms • 424 B • Save Response

Body: JSON

```
1 {
2   "details": [
3     "Email is required"
4   ],
5   "errorCode": "VALIDATION_ERROR",
6   "message": "Validation failed",
7   "success": false
8 }
```


[F1.3 — Login: Empty Request Body Validation](#)

Endpoint: POST /api/auth/login

Access: Public

Scenario: Submitting a login request with an empty JSON body.

Result: 400 Bad Request — The request is rejected with the `VALIDATION_ERROR` code, indicating that both email and password are required.

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8080/api/auth/login
- Body:** raw (selected), containing an empty JSON object:

```
{}
```
- Response:** 400 Bad Request (7 ms, 447 B). The response body is a JSON object:

```
{  "details": [    "Email is required",    "Password is required"  ],  "errorCode": "VALIDATION_ERROR",  "message": "Validation failed",  "success": false}
```

F1.5 — Profile Management: Successful Phone Update

Endpoint: PATCH /api/auth/me

Access: Authenticated User

Scenario: Updating only the phone number of the currently authenticated user.

Result: 200 OK — The phone number is updated successfully while all other profile fields remain unchanged.

The screenshot displays a REST client interface with a table at the top showing user data. Below the table, a PATCH request is configured to `http://localhost:8080/api/auth/me`. The request body is a JSON object with a `phone` field. The response is a 200 OK status with a JSON body containing user profile details and a success message.

FIRST_NAME	LAST_NAME	PASSWORD	PHONE	USER_STATUS	USER_ROLE
Omar	Ali	(bcrypt)\$2a\$10\$to.5y/1tJrU5eox5paHbzeuLvwJaByBbsHV2qcs8.y09duLDRS2Cm	01062861947	ACTIVE	CUSTOMER

Request: PATCH `http://localhost:8080/api/auth/me`

Body (JSON):

```
{
  "phone": "01012345678"
}
```

Response: 200 OK • 247 ms • 561 B

Body (JSON):

```
{
  "data": {
    "userId": 101,
    "firstName": "Omar",
    "lastName": "Ali",
    "email": "omar.profile@example.com",
    "phone": "01012345678",
    "role": "CUSTOMER",
    "userStatus": "ACTIVE",
    "airlineId": null
  },
  "message": "User profile updated successfully",
  "success": true
}
```

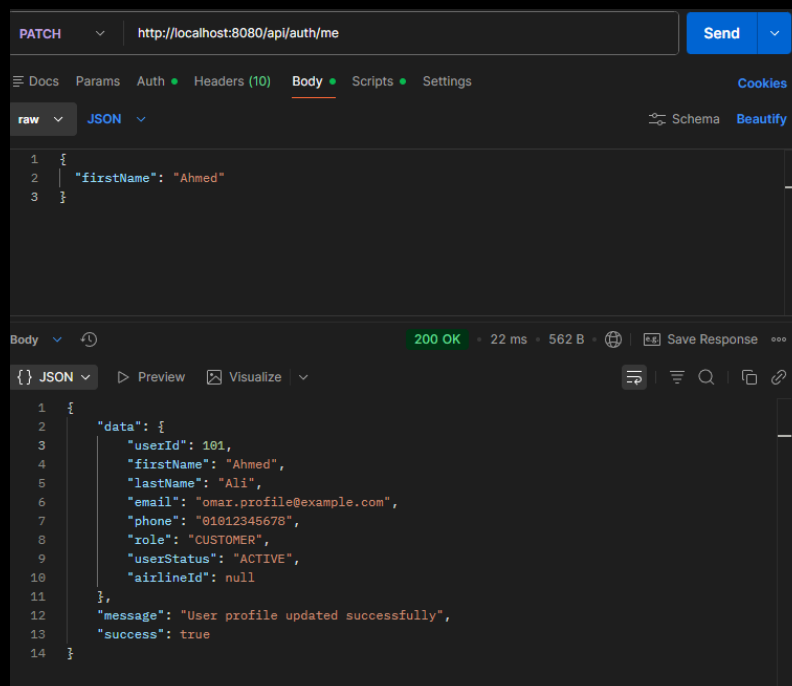
F1.5 — Profile Management: Successful First Name Update

Endpoint: PATCH /api/auth/me

Access: Authenticated User

Scenario: Updating only the first name of the currently authenticated user using a valid JWT access token.

Result: 200 OK — The first name is updated successfully while all other profile fields remain unchanged



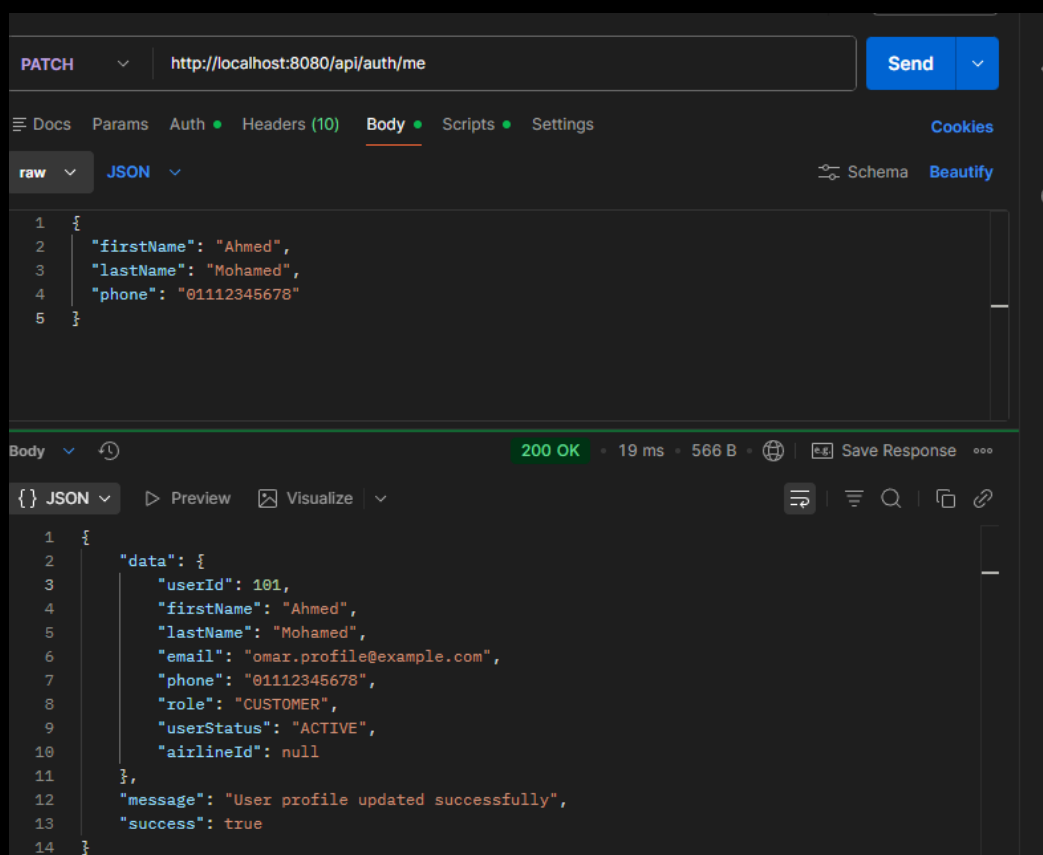
F1.5 — Profile Management: Successful Multiple-Field Update

Endpoint: PATCH /api/auth/me

Access: Authenticated User

Scenario: Updating all allowed profile fields—first name, last name, and phone number—in a single request using a valid JWT access token.

Result: 200 OK — All provided profile fields are updated successfully, while immutable fields such as email, role, user status, and airline ID remain unchanged.



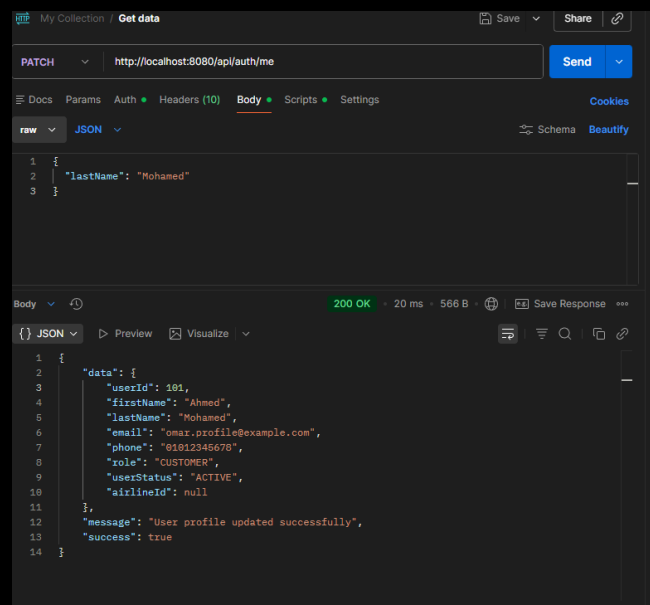
F1.5 – Profile Management: Successful Last Name Update

Endpoint: PATCH /api/auth/me

Access: Authenticated User

Scenario: Updating only the last name of the currently authenticated user using a valid JWT access token.

Result: 200 OK – The last name is updated successfully while all other profile fields remain unchanged.



F1.5 — Profile Management: Pre-Fix Unknown Field Handling

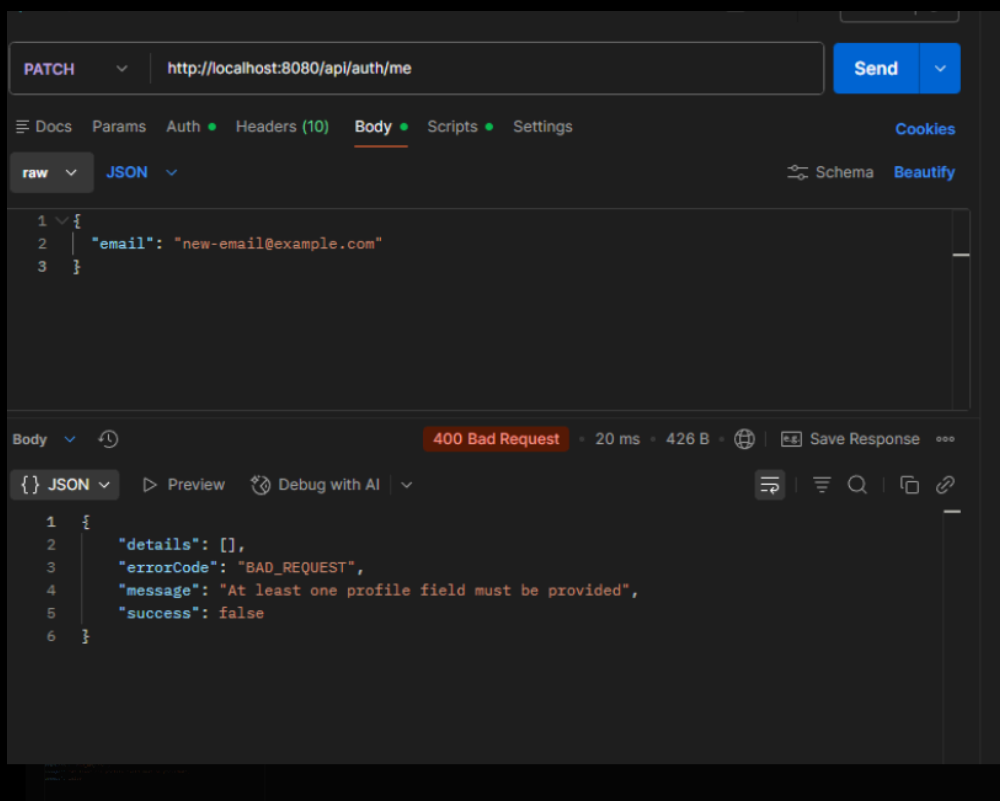
Endpoint: PATCH /api/auth/me

Access: Authenticated User

Scenario: Attempting to update the immutable email field before implementing the unknown-field exception handler.

Previous Result: 400 Bad Request — The request was incorrectly handled as an empty profile update and returned the BAD_REQUEST error code.

Issue: The API did not distinguish between an unsupported JSON field and an empty request containing no editable profile field



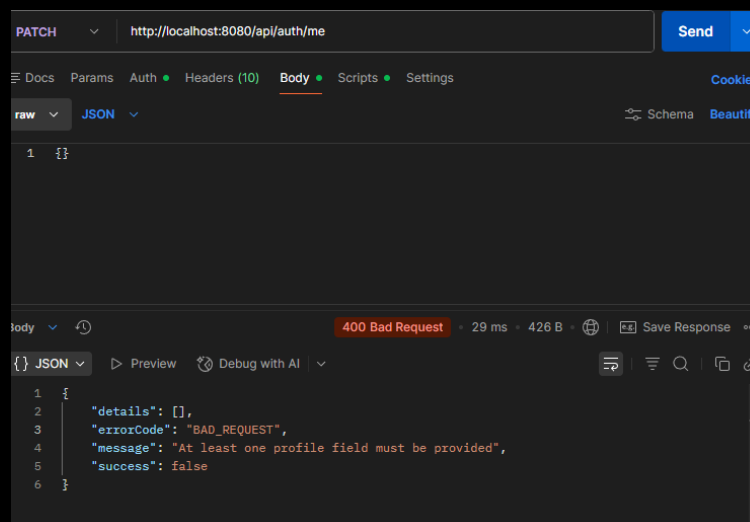
F1.5 – Profile Management: Empty Update Request Rejected

Endpoint: PATCH /api/auth/me

Access: Authenticated User

Scenario: Submitting an empty JSON object without providing any editable profile field.

Result: 400 Bad Request — The request is rejected with the BAD_REQUEST error code and the message “At least one profile field must be provided.”



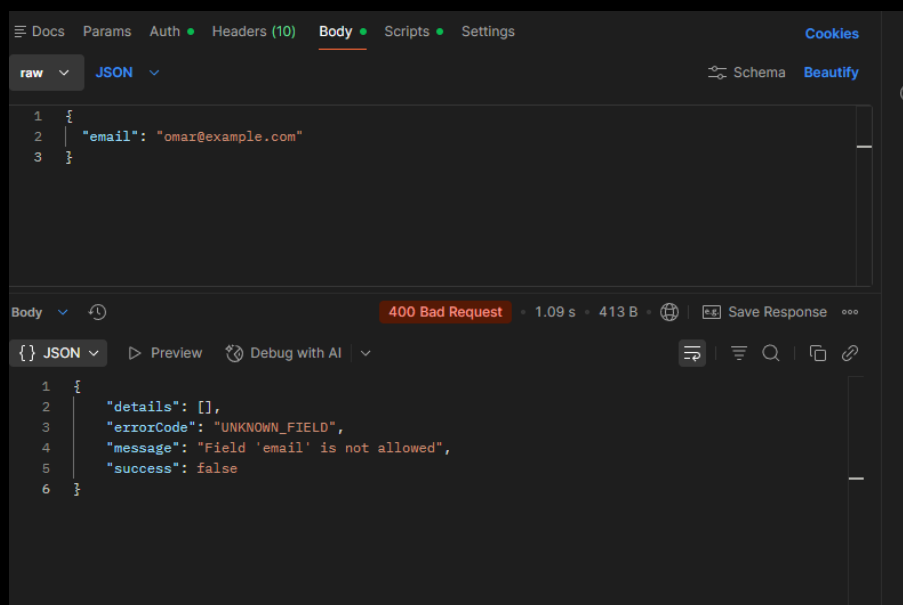
F1.5 – Profile Management: Unknown Field Rejected After Fix

Endpoint: PATCH /api/auth/me

Access: Authenticated User

Scenario: Attempting to update the email field after implementing the unknown-field exception handler.

Result: 400 Bad Request — The request is correctly rejected with the UNKNOWN_FIELD error code and the message “Field 'email' is not allowed”



F1.5 – Change Password: Successful Password Change

Endpoint: PUT /api/auth/change-password

Access: Authenticated User

Scenario: Changing the currently authenticated user's password by providing the correct old password and a valid new password.

Result: 200 OK — The old password is verified, the new password is securely encoded, and the password is changed successfully without exposing any password data in the response.

The screenshot displays a REST client interface with the following details:

- Method:** PUT
- URL:** http://localhost:8080/api/auth/change-password
- Buttons:** Send, Docs, Params, Auth, Headers (10), Body (selected), Scripts, Settings, Cookies, Schema, Beautify.
- Request Body (JSON):**

```
1 {
2   "oldPassword": "Password@123",
3   "newPassword": "NewPassword@123"
4 }
```
- Response Status:** 200 OK (1.45 s, 396 B)
- Response Body (JSON):**

```
1 {
2   "data": null,
3   "message": "Password changed successfully",
4   "success": true
5 }
```

F1.5 – Profile Management: Successful Profile Retrieval

Endpoint: GET /api/auth/me

Access: Authenticated User

Scenario: Retrieving the currently authenticated user's profile using a valid JWT access token.

Result: 200 OK — The current user profile is returned successfully without exposing the password or password hash.

